Министерство науки и высшего образования Российской Федерации

федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет
имени академика С.П. Королева»

Институт информатики и кибернетики

Кафедра технической кибернетики

Отчет по лабораторной работе №3

Дисциплина: «ООП»

Выполнил: Илларионов Е.А.

Группа: 6201-120303D

Самара, 2025

Задание 1

Изучил классы исключений Java:

java.lang.Exception

java.lang.IndexOutOfBoundsException

java.lang.ArrayIndexOutOfBoundsException

java.lang.IllegalArgumentException

java.lang.IllegalStateException

Задание 2

В пакете functions создал 2 класса исключений
FunctionPointIndexOutOfBoundsException

FunctionPointIndexOutOfBoundsException.java | InappropriateFunctionPointException

```
1  package functions;
2
3  public class FunctionPointIndexOutOfBoundsException
4          extends IndexOutOfBoundsException {
5
6      public FunctionPointIndexOutOfBoundsException() {
7          super();
8      }
9
10     public FunctionPointIndexOutOfBoundsException(String message) {
11         super(message);
12     }
13 }
14
```

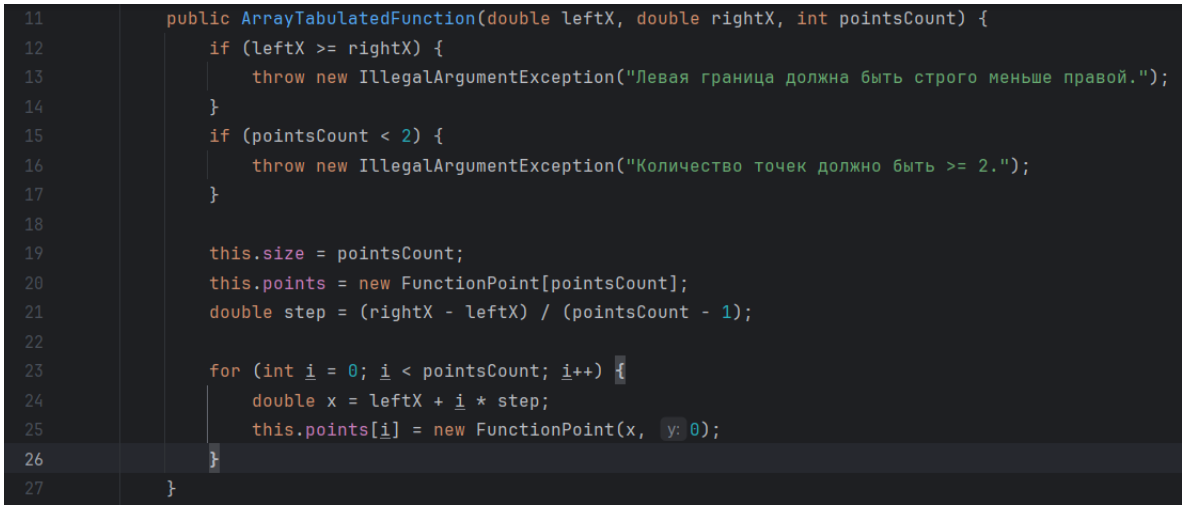
InappropiateFunctionPointExpection

FunctionPointIndexOutOfBoundsException.java | InappropriateFunctionPointException.jav

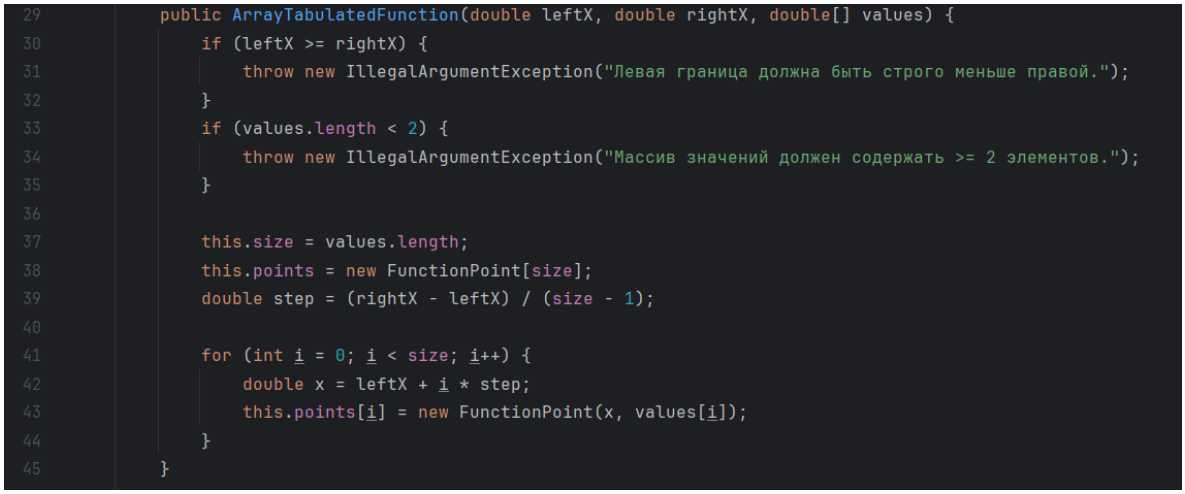
```
1  package functions;
2
3  public class InappropriateFunctionPointException extends Exception {
4
5      public InappropriateFunctionPointException() {
6          super();
7      }
8
9      public InappropriateFunctionPointException(String message) {
10         super(message);
11     }
12 }
13
```

Задание 3

В разработанный ранее класс TabulatedFunction внес изменения, обеспечивающие выбрасывание исключений методом класса



```
11      public ArrayTabulatedFunction(double leftX, double rightX, int pointsCount) {
12          if (leftX >= rightX) {
13              throw new IllegalArgumentException("Левая граница должна быть строго меньше правой.");
14          }
15          if (pointsCount < 2) {
16              throw new IllegalArgumentException("Количество точек должно быть >= 2.");
17          }
18
19          this.size = pointsCount;
20          this.points = new FunctionPoint[pointsCount];
21          double step = (rightX - leftX) / (pointsCount - 1);
22
23          for (int i = 0; i < pointsCount; i++) {
24              double x = leftX + i * step;
25              this.points[i] = new FunctionPoint(x,  y: 0);
26          }
27      }
```



```
29      public ArrayTabulatedFunction(double leftX, double rightX, double[] values) {
30          if (leftX >= rightX) {
31              throw new IllegalArgumentException("Левая граница должна быть строго меньше правой.");
32          }
33          if (values.length < 2) {
34              throw new IllegalArgumentException("Массив значений должен содержать >= 2 элементов.");
35          }
36
37          this.size = values.length;
38          this.points = new FunctionPoint[size];
39          double step = (rightX - leftX) / (size - 1);
40
41          for (int i = 0; i < size; i++) {
42              double x = leftX + i * step;
43              this.points[i] = new FunctionPoint(x, values[i]);
44          }
45      }
```

Методы getPoint(), getPointX(), getPointY(), setPointY() выбрасывают исключение FunctionPointIndexOutOfBoundsException, если переданный в метод номер выходит за границы набора точек.

Методы setPoint(), setPointX(), выбрасывают исключение FunctionPointIndexOutOfBoundsException, если переданный в метод номер выходит за границы набора точек, и исключение InappropiateFunctionPointExpection в том случае, если координата x задаваемой точки лежит вне интервала, определяемого значениями соседних точек табулированной функции.

Задание 4

В пакете functions создал класс LinkedListTabulatedFunction.

Класс FunctionNode реализован как приватный вложенный класс, обеспечивая инкапсуляцию структуры списка.

```
3   public class LinkedListTabulatedFunction implements TabulatedFunction {
4
5       // Вложенный приватный класс FunctionNode
6       private static class FunctionNode {
7           public FunctionPoint point;
8           public FunctionNode prev;
9           public FunctionNode next;
10
11          public FunctionNode(FunctionPoint point) {
12              this.point = point;
13              this.prev = this.next = this;
14          }
15      }
```

Реализован метод getNodeByIndex(int index), который использует кэширование последнего узла и выбирает направление поиска (от начала или от конца списка) для ускорения операций.

```
74      private FunctionNode getNodeByIndex(int index) {
75          FunctionNode current;
76
77          if (index == lastAccessedIndex) {
78              return lastAccessedNode;
79          }
80
81          if (index < size / 2) {
82              current = head.next;
83              for (int i = 0; i < index; i++) {
84                  current = current.next;
85              }
86          } else {
87              current = head.prev;
88              for (int i = size - 1; i > index; i--) {
89                  current = current.prev;
90              }
91          }
```

Задание 5

В классе LinkedListTabulatedFunction реализованы публичные конструкторы и методы, соответствующие сигнатурам ArrayTabulatedFunction

```java
public LinkedListTabulatedFunction(double leftX, double rightX, double[] values) {
    if (leftX >= rightX) {
        throw new IllegalArgumentException("Левая граница должна быть строго меньше правой.");
    }
    if (values.length < 2) {
        throw new IllegalArgumentException("Массив значений должен содержать >= 2 элементов.");
    }

    initializeList();

    double step = (rightX - leftX) / (values.length - 1);

    for (int i = 0; i < values.length; i++) {
        double x = leftX + i * step;
        addNodeToTail(new FunctionPoint(x, values[i]));
    }
}
```

```java
public double getRightDomainBorder() {
    if (size == 0) return Double.NaN;
    return head.prev.point.getX();
}

@Override
public double getFunctionValue(double x) {
    if (x < getLeftDomainBorder() || x > getRightDomainBorder()) return Double.NaN;

    FunctionNode current = head.next;
    if (lastAccessedIndex != -1) {
        current = lastAccessedNode;
    }

    while (current.next != head) {
        FunctionNode nextNode = current.next;

        if (x >= current.point.getX() && x <= nextNode.point.getX() + EPSILON) {
            lastAccessedNode = current;
            lastAccessedIndex = -1;
            // Линейная интерполяция
            double x0 = current.point.getX();
            double y0 = current.point.getY();
            double x1 = nextNode.point.getX();
            double y1 = nextNode.point.getY();

            if (Math.abs(x1 - x0) < EPSILON) return y0;

            return y0 + (y1 - y0) * (x - x0) / (x1 - x0);
        }
```

```
196             current = current.next;
197         }
198         return Double.NaN;
199     }
200
201     @Override
202     public int getPointsCount() {
203         return size;
204     }
205
206     @Override
207     public FunctionPoint getPoint(int index) {
208         checkIndex(index);
209         return new FunctionPoint(getNodeByIndex(index).point);
210     }
211
212     @Override
213     public void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException {
214         checkIndex(index);
215
216         FunctionNode targetNode = getNodeByIndex(index);
217         double newX = point.getX();
218
219         if (targetNode.prev != head && newX < targetNode.prev.point.getX() + EPSILON) {
220             throw new InappropriateFunctionPointException("Новая абсцисса должна быть строго больше предыду
221         }
222         if (targetNode.next != head && newX > targetNode.next.point.getX() - EPSILON) {
223             throw new InappropriateFunctionPointException("Новая абсцисса должна быть строго меньше следующ
224         }
225
226         targetNode.point.setX(newX);
227         targetNode.point.setY(point.getY());
228     }
229
230     @Override
231     public double getPointX(int index) {
232         checkIndex(index);
233         return getNodeByIndex(index).point.getX();
234     }
235
236     @Override
237     public void setPointX(int index, double x) throws InappropriateFunctionPointException {
238         checkIndex(index);
239
240         FunctionNode targetNode = getNodeByIndex(index);
```

```
241
242             if (targetNode.prev != head && x < targetNode.prev.point.getX() + EPSILON) {
243                 throw new InappropriateFunctionPointException("Новая абсцисса должна быть строго бо
244             }
245             if (targetNode.next != head && x > targetNode.next.point.getX() - EPSILON) {
246                 throw new InappropriateFunctionPointException("Новая абсцисса должна быть строго ме
247             }
248             targetNode.point.setX(x);
249         }
250
251         @Override
252         public double getPointY(int index) {
253             checkIndex(index);
254             return getNodeByIndex(index).point.getY();
255         }
256
257         @Override
258         public void setPointY(int index, double y) {
259             checkIndex(index);
260             getNodeByIndex(index).point.setY(y);
261         }
262
```

```
263     @Override
264     public void deletePoint(int index) {
265         if (size <= 2) {
266             throw new IllegalStateException("Невозможно удалить точку: количество точек должно быть не мен
267         }
268
269         checkIndex(index);
270         deleteNodeByIndex(index);
271     }
272
273     @Override
274     public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException {
275
276         FunctionNode current = head.next;
277         int pos = 0;
278
279         while (current != head) {
280             if (Math.abs(current.point.getX() - point.getX()) < EPSILON) {
281                 throw new InappropriateFunctionPointException("Точка с такой абсциссой уже существует: " +
282             }
283             if (current.point.getX() > point.getX()) {
284                 break;
285             }
286             current = current.next;
287             pos++;
288         }
289
290         addNodeByIndex(pos, point);
291     }
```

Задание 6

Класс TabulatedFunction переименован в ArrayTabulatedFunction.

Создан интерфейс TabulatedFunction.java, содержащий объявления всех общих методов.

Оба класса функций (ArrayTabulatedFunction и LinkedListTabulatedFunction) реализовали созданный интерфейс.

Задание 7

Класс main

```java
import functions.FunctionPoint;
import functions.TabulatedFunction;
import functions.ArrayTabulatedFunction;
import functions.LinkedListTabulatedFunction;
import functions.FunctionPointIndexOutOfBoundsException;
import functions.InappropriateFunctionPointException;

public class Main {

    // Вспомогательный метод для тестирования исключений
    public static void testFunction(TabulatedFunction func, String name) {
        System.out.println("\n--- Тестирование исключений для " + name + " ---");

        // 1. IllegalArgumentException (Проверка конструкторов)
        try {
            new ArrayTabulatedFunction( leftX: 10,  rightX: 0,  pointsCount: 5); // leftX >= rightX
        } catch (IllegalArgumentException e) {
            System.out.println("Конструктор (OK): " + e.getMessage());
        }

        // 2. IllegalStateException (Проверка deletePoint при size <= 2)
        try {
            double[] values = {1, 2};
            TabulatedFunction shortFunc = new ArrayTabulatedFunction( leftX: 0,  rightX: 1, values);
            shortFunc.deletePoint( index: 0);
        } catch (IllegalStateException e) {
            System.out.println("deletePoint (OK): " + e.getMessage());
        }

        // 3. FunctionPointIndexOutOfBoundsException
        try {
            func.getPoint( index: 100);
        } catch (FunctionPointIndexOutOfBoundsException e) {
            System.out.println("getPoint (OK): " + e.getMessage());
        }

        // 4. InappropriateFunctionPointException (Проверка addPoint - совпадение X)
        try {
            func.addPoint(new FunctionPoint(func.getPointX( index: 1),  y: 50));
        } catch (InappropriateFunctionPointException e) {
            System.out.println("addPoint (OK): " + e.getMessage());
        } catch (Exception e) {
            // Исключения, не связанные с логикой addPoint
        }

        // 5. InappropriateFunctionPointException (Проверка setPointX - нарушение упорядоченности)
        try {
            // Пытаемся сделать X[1] меньше X[0]
            func.setPointX( index: 1,  x: func.getPointX( index: 0) - 1);
        } catch (InappropriateFunctionPointException e) {
            System.out.println("setPointX (OK): " + e.getMessage());
        } catch (Exception e) {
            // Исключения, не связанные с логикой setPointX
        }
```

```
55
56          System.out.println("Тестирование завершено.");
57      }
58
59      public static void main(String[] args) {
60          double[] values = {10, 20, 30, 40, 50};
61
62          // Используем интерфейсный тип для объявления переменной
63          TabulatedFunction arrayFunc = new ArrayTabulatedFunction( leftX: 0,  rightX: 4, values);
64          TabulatedFunction linkedFunc = new LinkedListTabulatedFunction( leftX: 0,  rightX: 4, values);
65
66          System.out.println("--- Проверка работы ArrayTabulatedFunction ---");
67          System.out.println("Значение в X=1.5: " + arrayFunc.getFunctionValue( x: 1.5));
68
69          System.out.println("\n--- Проверка работы LinkedListTabulatedFunction ---");
70          System.out.println("Количество точек: " + linkedFunc.getPointsCount());
71          System.out.println("Правая граница: " + linkedFunc.getRightDomainBorder());
72          System.out.println("Значение в X=2.5: " + linkedFunc.getFunctionValue( x: 2.5));
73
74          // Тестируем исключения
75          testFunction(arrayFunc,  name: "ArrayTabulatedFunction");
76          testFunction(linkedFunc,  name: "LinkedListTabulatedFunction");
77      }
78  }
```